

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING

COURSE CODE: CSA4303

Name : T. Nikhil Kumar Reddy

Reg-No: 192372024

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. (L2)

Assessment Tool : Data Interpretation

Weightage: 20%

QUESTIONS:

Total Marks: 20

1: HTTP Response Analysis

A web server returns the following HTTP response header:

HTTP/1.1 200 OK

Date: Mon, 01 Apr 2026 10:00:00 GMT

Content-Type: text/html

Content-Length: -150

Connection: keep-alive

Questions:

1. Identify any incorrect or invalid fields in the header.
2. Analyze the issue with Content-Length and explain its impact.
3. Determine whether the status code matches the response correctness.
4. Suggest corrections for the identified errors.

1. Identify incorrect/invalid fields in the header

HTTP/1.1 200 OK

Date: Mon, 01 Apr 2026 10:00:00 GMT

Content-Type: text/html

Content-Length: -150

Connection: keep-alive

- The invalid field is **Content-Length: -150**. Content-Length must be a non-negative integer representing the number of bytes in the response body. A negative value is syntactically and semantically invalid.
- All other fields (Date, Content-Type, Connection) are correctly formatted.

2. Analyze the issue with Content-Length and explain its impact

- Content-Length tells the client exactly how many bytes to expect in the body, so it knows when the response ends.
- A negative value is impossible in reality — it violates the HTTP/1.1 specification (RFC 7230/9110), which requires it to be a valid decimal digit sequence ≥ 0 .
- **Impact:**
 - The browser/client cannot determine the correct body size, leading to parsing errors.
 - The connection may hang, get truncated, or be forcibly closed since Connection: keep-alive depends on knowing exact content boundaries.
 - Some clients may reject the response entirely as malformed, causing the page to fail to load.

3. Determine whether the status code matches the response correctness

- The status line is 200 OK, which indicates a successful request.
- However, the response is **not actually correct** because the header itself is malformed (invalid Content-Length).
- So while the status code claims success, the response is internally inconsistent — a 200 status should only accompany a well-formed, deliverable body. This is a mismatch between the claimed status and actual validity of the response.

4. Suggested corrections

HTTP/1.1 200 OK

Date: Mon, 01 Apr 2026 10:00:00 GMT

Content-Type: text/html

Content-Length: 150

Connection: keep-alive

- Replace -150 with the actual, correct positive byte count of the response body (e.g., 150).

- Optionally, the server should validate Content-Length dynamically (computed from actual body size) rather than hardcoding it.

2: DOM Structure Analysis

Consider the following DOM tree representation:

```
<html>
  <head>
    <title>Test Page</title>
  </head>
  <body>
    <div>
      <p>Hello World
    </div>
  </body>
</html>
```

Questions:

1. Identify missing or improperly closed elements.
2. Analyze how the browser will interpret this structure.
3. Determine the impact on rendering and layout.
4. Suggest corrections to make the DOM valid.

Question 2: DOM Structure Analysis

Given:

html

```
<html>

  <head>

    <title>Test Page</title>

  </head>

  <body>

    <div>

      <p>Hello World

    </div>

  </body>

</html>
```

1. Identify missing or improperly closed elements

- `<p>Hello World` — the `<p>` tag is **never closed** with `</p>`.
- `<div>` is closed with `</div>`, but this closing occurs before the paragraph tag is closed — causing improper nesting.

2. Analyze how the browser will interpret this structure

- HTML parsers follow **auto-closing/error-recovery rules**. Since `<p>` is not explicitly closed, the browser will automatically close it when it encounters the `</div>` tag (because `<p>` cannot legally contain a block-level closing boundary like `</div>`).
- Effectively, the browser's parser will insert an implicit `</p>` right before `</div>`, producing a corrected internal DOM tree even though the source HTML is invalid.

3. Determine the impact on rendering and layout

- In this specific case, since only one `<p>` is involved, the browser's error correction usually **recovers gracefully**, and the visual output looks correct ("Hello World" displays inside the div).
- However:
 - Relying on browser auto-correction is unpredictable across different browsers/parsers.
 - If more nested or sibling elements existed, mis-closing could cause elements to be misplaced in the DOM tree, leading to broken layout, incorrect CSS styling (since selectors depend on DOM structure), and possible JavaScript DOM traversal errors.
 - It also fails HTML validation (e.g., W3C validator), affecting standards compliance.

4. Suggested corrections

html

`<html>`

`<head>`

`<title>Test Page</title>`

`</head>`

`<body>`

`<div>`

`<p>Hello World</p>`

`</div>`

`</body>`

</html>

- Add the missing </p> closing tag before </div> to ensure valid, properly nested markup.

3: GET vs POST in Login System

A login system uses two different methods:

Method	URL Example	Data Visibility	Security Level
GET	/login?user=admin&pass=1234	Visible in URL	Low
POST	/login	Hidden in body	Moderate

Questions:

1. Interpret the differences in data transmission between GET and POST.
2. Analyze which method is more suitable for login and why.
3. Identify security risks in using GET for authentication.
4. Suggest best practices for secure login implementation.

Question 3: GET vs POST in Login System

1. Interpret the differences in data transmission

Aspect	GET	POST
Data location	Appended to URL as query string (?user=admin&pass=1234)	Sent in the HTTP request body
Visibility	Visible in browser address bar, browser history, server logs	Hidden from URL, not shown in address bar
Caching	Can be cached by browsers/proxies	Typically not cached
Data length	Limited by URL length restrictions	Can carry larger amounts of data
Bookmarking	URL (with data) can be bookmarked	Cannot be bookmarked with data

2. Analyze which method is more suitable for login and why

- **POST is more suitable** for login systems because:
 - Credentials are sent in the request body, not exposed in the URL.
 - Data doesn't get stored in browser history or server access logs.
 - It avoids accidental leakage via shared/bookmarked links or referrer headers.

3. Identify security risks in using GET for authentication

- Username and password appear directly in the URL, which:

- Gets logged in server logs, proxy logs, and browser history in plaintext.
- Can be leaked via the Referer header to third-party sites if a link is clicked from the page.
- Can be shared accidentally (e.g., copy-pasting a URL) exposing credentials.
- Is more susceptible to shoulder-surfing or screen-sharing exposure.
- Can be cached by browsers or intermediate proxies, increasing exposure duration.

4. Suggested best practices for secure login implementation

- Always use **POST** for transmitting login credentials, never GET.
- Use **HTTPS (TLS/SSL)** to encrypt data in transit, since POST alone doesn't encrypt data — it only avoids URL exposure.
- Implement **hashing/salting** of passwords server-side (never send/store plaintext long-term).
- Use **secure, HttpOnly, SameSite cookies** for session management after login.
- Apply **rate limiting/CAPTCHA** to prevent brute-force attacks.
- Consider **multi-factor authentication (MFA)** for additional security.

4: HTML Code Inspection

Analyze the following HTML snippet:

```
<!DOCTYPE html>
<html>
<head>
<title>Sample</title>
</head>
<body>
<h1>Welcome
<p>This is a paragraph

</body>
</html>
```

Questions:

1. Identify missing closing tags.
2. Analyze issues with the tag.
3. Determine how browsers will handle these errors.
4. Suggest a corrected version of the code.

1. Identify missing closing tags

- `<h1>Welcome` — missing `</h1>`
- `<p>This is a paragraph` — missing `</p>`
- (`<img>` is a void/self-closing element, so it doesn't need a closing tag — see below.)

2. Analyze issues with the `<img>` tag

- `` is missing the **alt attribute**, which is important for:
 - Accessibility (screen readers describe the image using alt text).
 - SEO (search engines use alt text to understand image content).
 - Fallback text display if the image fails to load.
- In strict **XHTML**, void elements must be self-closed: ``. In plain HTML5, `<img>` without a closing slash is valid, but for standards compliance (especially XHTML as per CO-1), it should be self-closed.

3. Determine how browsers will handle these errors

- Browsers use **error-recovery/auto-correction** parsing:
 - `<h1>` will be auto-closed when the next block-level element (`<p>`) starts.
 - `<p>` will be auto-closed when `</body>` is encountered.
 - The missing alt attribute won't break rendering, but it degrades accessibility and standards compliance.
- Visually, the page may still render correctly in most browsers, but the document will **fail HTML/XHTML validation** and behave unpredictably in edge cases or with assistive technologies.

4. Suggested corrected version

html

`<!DOCTYPE html>`

`<html>`

`<head>`

`<title>Sample</title>`

`</head>`

`<body>`

`<h1>Welcome</h1>`

```
<p>This is a paragraph</p>



</body>

</html>
```

5: Browser-Server Communication Flow

The following sequence describes a web request:

1. User enters a URL in the browser
2. DNS lookup occurs
3. HTTP request is sent
4. Server processes request
5. Response is returned
6. Browser renders page

Questions:

1. Interpret each step in the communication flow.
2. Identify where delays can occur.
3. Analyze the role of DNS in the process.
4. Suggest optimizations to improve performance.

1. Interpret each step in the communication flow

1. **User enters a URL** — The browser parses the URL into protocol, domain, path, and query components.
2. **DNS lookup** — The domain name is resolved into an IP address using the Domain Name System.
3. **HTTP request sent** — The browser establishes a TCP connection (often with TLS handshake for HTTPS) and sends an HTTP request (e.g., GET) to the resolved server IP.
4. **Server processes request** — The server (web server + backend application/database, if needed) processes the request and prepares a response.
5. **Response returned** — The server sends back an HTTP response with status code, headers, and body (HTML/JSON/etc.).
6. **Browser renders page** — The browser parses HTML, builds the DOM, applies CSS (CSSOM), executes JavaScript, and paints the final page.

2. Identify where delays can occur

- **DNS resolution delay** — if DNS cache is cold or DNS server is slow.
- **TCP/TLS handshake delay** — especially for HTTPS connections with certificate negotiation.
- **Server processing delay** — slow backend logic, database queries, or server overload.
- **Network latency/bandwidth** — physical distance between client and server, congestion.
- **Rendering delay** — large HTML/CSS/JS files, render-blocking resources, or heavy DOM manipulation on the client side.

3. Analyze the role of DNS in the process

- DNS acts as the "**phonebook of the internet**", translating human-readable domain names (e.g., www.example.com) into machine-readable IP addresses.
- Without DNS resolution, the browser cannot know which server to connect to.
- DNS lookup time directly adds to the overall page load time — this is why DNS caching (browser, OS, or ISP-level) is critical for performance.

4. Suggested optimizations to improve performance

- **DNS-level:** Use DNS caching, DNS prefetching (<link rel="dns-prefetch">), or reliable/fast DNS providers.
- **Connection-level:** Use HTTP/2 or HTTP/3, persistent (keep-alive) connections, and Content Delivery Networks (CDNs) to reduce latency.
- **Server-level:** Optimize backend processing, use caching (server-side/database caching), and load balancing.
- **Content-level:** Minify HTML/CSS/JS, compress responses (gzip/Brotli), lazy-load images, and reduce render-blocking resources.
- **Client-level:** Use browser caching (via appropriate Cache-Control headers) to avoid repeated full downloads.

Code of Conduct:

I T.Nikhil Kumar Reddy / 192372024 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: T. Nikhil Kumar Reddy

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Data	25%	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding; misses key trends	Misinterprets or fails to understand data
Analysis	25%	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some justification	Basic analysis with weak reasoning	No meaningful analysis provided
Application of Concepts	20%	Effectively applies relevant concepts/tools to interpret data accurately	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts to data
Conclusion	20%	Draws clear, meaningful conclusions with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions
Clarity	10%	Presents interpretation clearly using proper structure, visuals, and terminology	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand